



SnapNote v1.6.0

更新机制重构产品需求文档

双目录原子切换 · 从源头消灭更新链路的结构性风险

版本 | v1.6.0 (架构重构版)

日期 | 2026-10-05

状态 | 待确认 (确认后开工)

决策 | 方案 A: 先立地基, 浏览器扩展二期顺延 v1.7.0



目录

目录	2
一、一句话目标	3
二、为什么必须重构	3
三、新架构总览	3
3.1 目录布局	3
3.2 角色职责	3
四、四条关键链路设计	4
4.1 全新安装	4
4.2 常规更新 (v1.6.0+, 核心链路, 100% Node)	4
4.3 v1.5.x → v1.6.0 迁移 (旧 PS 链最后一跑, 接力式)	4
4.4 回滚 (用户主动权)	4
五、关键决策与取舍	4
六、测试与验收标准	5
6.1 自动化 (CI 全绿才发版)	5
6.2 真机验收 (两跳, 缺一不可)	5
七、风险清单与兜底	5
八、工作量与里程碑	5
附：与现状的兼容性承诺	6

一、一句话目标

把更新机制从「程序退出后由外部脚本删旧拷新（危险、不可测、环境依赖）」重构为「新版本完整就位后翻指针（安全、可测、环境无关）」——从源头消灭 tar 劫持、robocopy 中断、杀软拦截这一整类故障，让以后每个版本的发布都不再是赌注。

二、为什么必须重构

v1.5.x 替换链是结构性缺陷，补丁无法收敛。证据如下：

证据	说明
5/7 版本在修同一条链路	同一子系统反复出问题 = 结构问题而非实现问题
失败空间无限 vs 测试空间有限	PS 脚本跑在用户环境全集（PATH/杀软/盘符/权限），CI 只能模拟有限环境
破坏性语义	robocopy /MIR 先删后拷，任何中途死亡 = 目录半损
成熟度对照组	Chrome/Steam 均为「双目录+指针」；无主流软件以「退出后脚本镜像替换」为唯一更新路径

v1.5.5 的多层防御（绝对路径 tar、兜底解压、回滚、留痕）把已知坑埋了，但「不再踩新坑」无法被证明。本重构消灭的是坑的类别。

三、新架构总览

3.1 目录布局

E:\project\SnapNote\	← 解压根目录（双击入口永远在这层）
├ SnapNote.exe	← 启动器（极薄 stub，约 10KB，逻辑永不变化）
├ channels.json	← 指针：{"current":"app-1.6.0","previous":null}
├ app-1.6.0\	← 版本目录 A（当前运行）
│ ├── SnapNoteApp.exe	← 主程序（v1.6.0 起改名，原因见决策 1）
│ ├── resources\	← Electron 资源
│ └ ...	
└ app-1.7.0\	← 版本目录 B（更新就位后；A 变为回滚后路）

3.2 角色职责

角色	形态	职责	更新频率
启动器 SnapNote.exe	C# stub（源码入仓库，CI 编译）	读指针拉起主程序；指针损坏时扫 app-* 取最大版本兜底；全损弹窗	永不更新
主程序 SnapNote App.exe	Electron 应用	便签/收集/气泡/更新器宿主，更新器全程 Node	每版本
channels.json	纯文本指针	current / previous 两个版本目录名	每次更新翻转

四、四条关键链路设计

4.1 全新安装

下载 zip（新布局成品）→ 解压到任意目录 → 双击根 SnapNote.exe（启动器）→ 读 channels.json → 拉起 app-1.6.0\SnapNoteApp.exe。解压即用，与现状体验完全一致。

4.2 常规更新（v1.6.0+，核心链路，100% Node）

下载 zip + sha256 校验（现有 27 个测试继续护住，代码不动）
→ applyUpdate（全 Node，可单测）：
1. System32 bsdtar 绝对路径解压到临时目录（失败无害可重试）
2. zip 内 app-x.y.z\ 挪入根（同卷 rename 原子；跨卷 copy+校验+删源）
3. 指针翻转（先写临时文件再 rename，原子保证）
4. 托盘通知「新版本已就绪」——不自动杀程序，主动权在用户
→ 用户点「重启进新版」→ spawn app-x.y.z\SnapNoteApp.exe → 本进程退出
→ 新版本首跑：异步清理 previous 目录 + 临时残留

失败语义：任何一步失败 = 旧版本原地照跑，指针未翻，用户零感知损失。最坏情况是「这次更新没成」，而不是旧版任何形式的损坏。

4.3 v1.5.x → v1.6.0 迁移（旧 PS 链最后一跑，接力式）

现有 v1.5.5 客户端执行未修改的 PS 脚本对新布局 zip，逐行推演已验证成立：

PS 步骤	对新 zip 的执行结果
等待旧进程退出	Get-Process SnapNote ✓（旧真程序名未变）
System32 tar 解压	产出 {启动器, channels.json, app-1.6.0\SnapNoteApp.exe + resources}
robocopy /MIR /XF SnapNote.exe	app-1.6.0\ 整体镜像入根 ✓（主程序已改名，不触发 /XF 深层排除——改名的根本原因）；根启动器被 /XF 跳过，由下一步补
exe 两步换名	旧根 SnapNote.exe → .old；拷启动器 → 根 ✓
启动根 exe	= 启动器 → 读指针 → 拉起 app-1.6.0\SnapNoteApp.exe ✓
新程序首跑	cleanupStale 删 .old ✓ 迁移完成

旧链一次都不用改，跑完即完成新布局铺设；此后该客户端永久转入 4.2 纯 Node 链路。v1.6.0 起删除 PS 生成逻辑（git 历史永久可回溯）。

4.4 回滚（用户主动权）

托盘菜单新增「回滚到上一版本」：指针翻转 → 经启动器拉起 previous 版本 → 通知「已回滚至 vX.Y.Z」。任何一次更新后不满意，10 秒内可退回，无需联系开发者。

五、关键决策与取舍

#	决策	理由	代价
1	主程序改名 SnapNoteApp.exe	旧 PS 的 /XF 按文件名深层排除同名 exe；改名后接力迁移零修改成立	任务管理器进程名显示 SnapNoteApp
2	启动器用 C# stub（CI csc 编译）	极薄秒启动，无 Electron 负担，不引入新构建依赖	仓库新增 launcher.cs（本地不编译，CI 负责）
3	解压暂用系统 bsdtar 绝对路径	零新依赖；失败语义已从致命降级为可重试	环境依赖残留一项（v1.7 换纯 JS 归零）
4	保留 previous 一层版本目录	回滚能力的物理基础	磁盘约 2× 版本目录；托盘提供「清理旧版本」
5	更新就绪后不自动重启	主动权交到用户手里	无（通知+菜单高亮引导）

六、测试与验收标准

6.1 自动化（CI 全绿才发版）

新增单测约 15 个：指针翻转原子性、双目录就位、跨卷拷贝分支、回滚翻转、新程序首跑清理、迁移识别、启动器目标推导。e2e_update.js 重写为双目录版，CI 真实跑「v1.6.0 → v1.6.1」全链——预期首次绿（不再有 PS 环境依赖）。冒烟改为启动器链：launcher → SnapNoteApp --smoke-test。

6.2 真机验收（两跳，缺一不可）

① 迁移跳：v1.5.5 真机点「重启并更新」升 v1.6.0 → 根目录出现启动器 + app-1.6.0 子目录 → 版本号 1.6.0 ✓（同时验证 v1.5.5 的 PS 修复，一箭双雕）。② Node 链跳：随即发布 v1.6.1 → 真机走纯 Node 更新 → 成功后回滚菜单可退回 1.6.0 ✓。两跳通过后，更新机制从「每次发布的赌注」转为「有测试护城河的常规代码」。

七、风险清单与兜底

风险	概率	兜底
启动器被 SmartScreen/杀软误报	低	与现状 zip 解压 exe 同属 MotW 场景，非新增类别；误报率极高时再评估签名
E 盘特殊文件系统 rename 非原子	低	指针用「临时文件+替换」双保险；失败重试无害
迁移跳失败（PS 最后一跑挂）	低	失败=停留 v1.5.5 无损；update-error.log 留痕；可手动解压 v1.6.0 zip 覆盖
启动器自身需更新的极端场景	极低	随版本目录分发新启动器副本，翻指针前同步替换根启动器（附加步骤非主路径）

八、工作量与里程碑



阶段	内容	估算
M1	launcher.cs + CI 编译与 zip 双目录组装 + 主程序改名	0.5 天
M2	Node 更新链 (applyUpdate/指针/重启/清理) + 15 个单测	1 天
M3	迁移验证 + CI e2e 重写 + 回滚菜单 + 失败留痕适配	0.5 天
M4	v1.6.0 发布 + 真机两跳验收 + v1.6.1 验证版	0.5 天 + 验收等待

总计约 2.5 个开发日。浏览器扩展二期（PRD 已定稿）顺延为 v1.7.0，其剪贴板/备忘录一期功能不受本次重构任何影响。

附：与现状的兼容性承诺

便签/备忘录/气泡/快捷键/磁吸零改动（87 测试继续全绿为准入线）；用户数据目录零迁移（%AppData% 与 exe 位置无关）；更新检查/下载（镜像、续传、sha256、截断检测）代码不动只换末端；v1.5.x 全系旧客户端可经 4.3 一步迁入新架构，错过多版无窗口期问题。